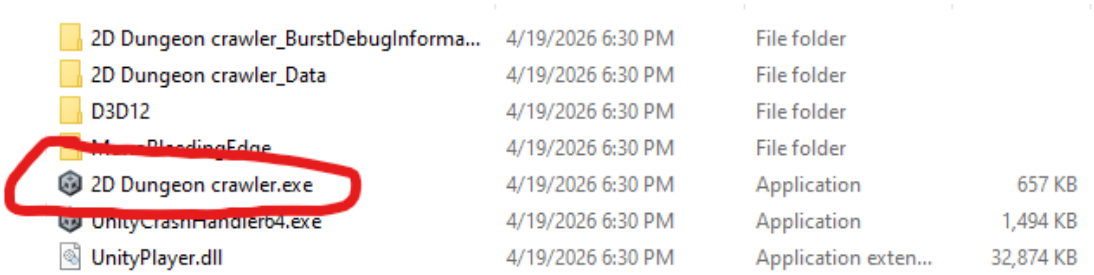
How to run the program:

First download the zip file from my website
This contains both the project and files necessary to run it. Export to a folder, keep all the files together, they are ALL necessary to run it.
Next, open 2D Dungeon Crawler



2D Dungeon crawler_BurstDebugInforma...	4/19/2026 6:30 PM	File folder	
2D Dungeon crawler_Data	4/19/2026 6:30 PM	File folder	
D3D12	4/19/2026 6:30 PM	File folder	
[illegible]	4/19/2026 6:30 PM	File folder	
2D Dungeon crawler.exe	4/19/2026 6:30 PM	Application	657 KB
UnityCrashHandler64.exe	4/19/2026 6:30 PM	Application	1,494 KB
UnityPlayer.dll	4/19/2026 6:30 PM	Application exten...	32,874 KB

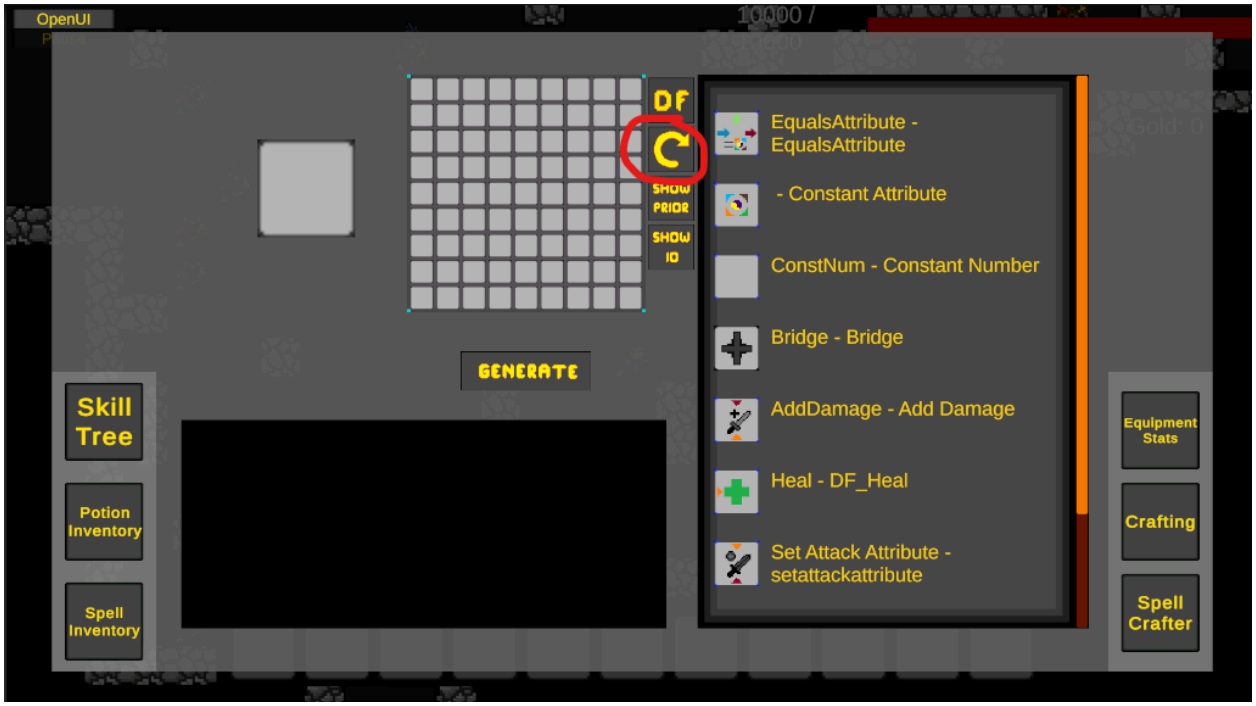
HOW TO USE THE SPELL SYSTEM:

First open the UI menu.

Open the spell crafter:

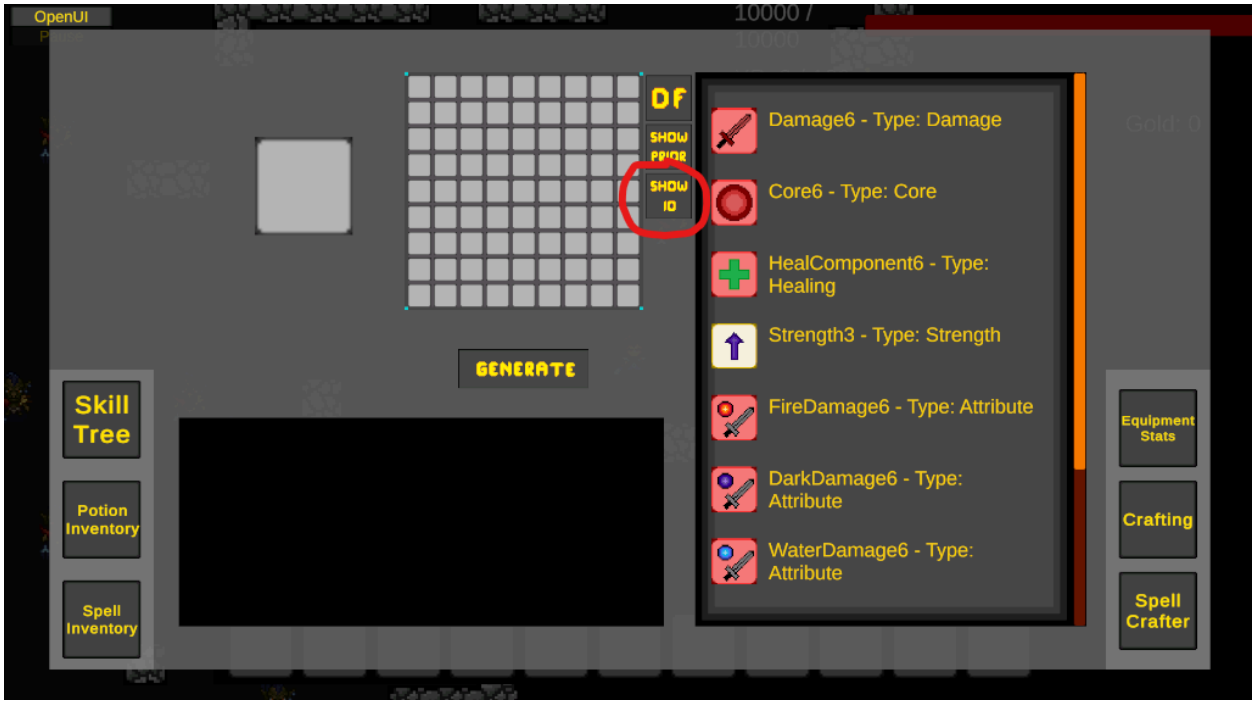
I'll go over the buttons now. First is the DF button. It opens the DF spell component menu.

If you are in the DF spell component menu, you will see a rotate button. This rotates components. DF components are particular about their direction. More on this later.



Next is the Show Prior button. This shows prior, weaker versions of the regular components. Everything is unlocked by default. You can swap back to max level components by pressing it again.

Next is the Show IO button. This shows the inputs and outputs for components. This is only really useful for DF components, as regular components can be any orientation. More on this later.

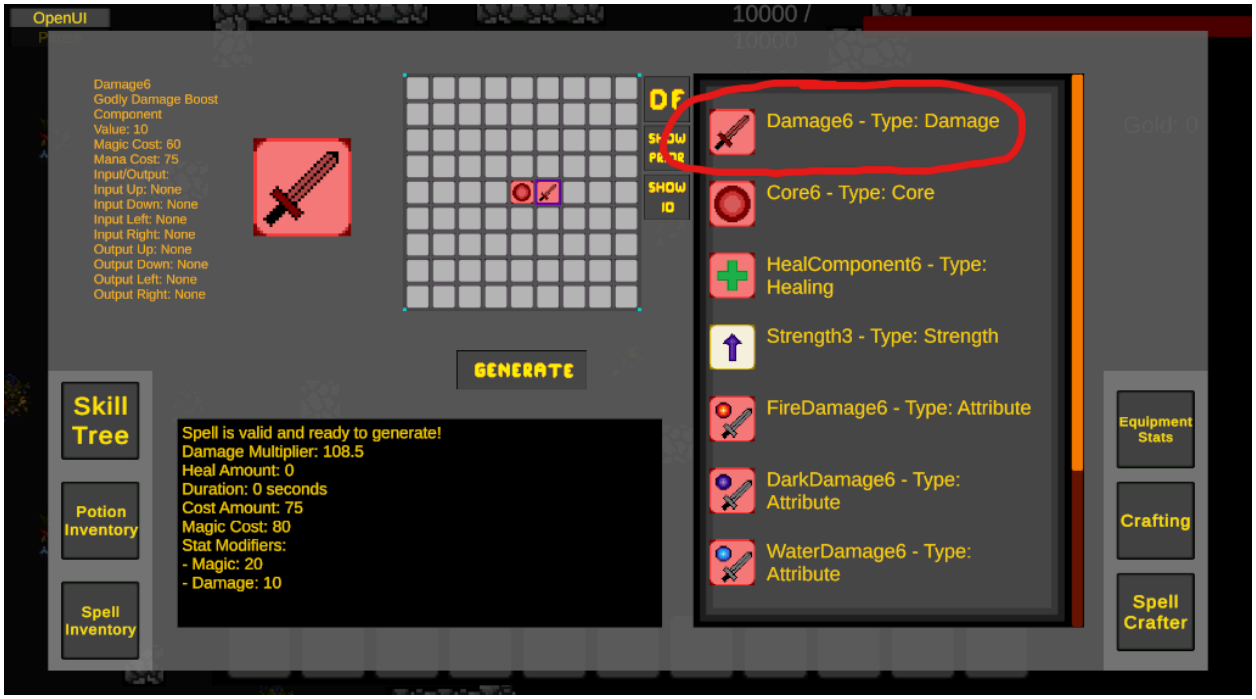


Here's how to make a basic damage spell. First, click on the circled cell to select which cell you want to place the component in.

Next, click on the core component. All spells MUST have a core, and can only have one core. Clicking the core component will place it in the selected cell.

Next, select a cell directly adjacent to the core component. Each component can only act on directly adjacent components. Some components cannot be placed next to certain other components. If you are clicking on a component and it is not placing it, that is likely the cause. To remove a component, right click on the cell it is in. See bottom for detailed component guide.

Next, select the “Damage” component.



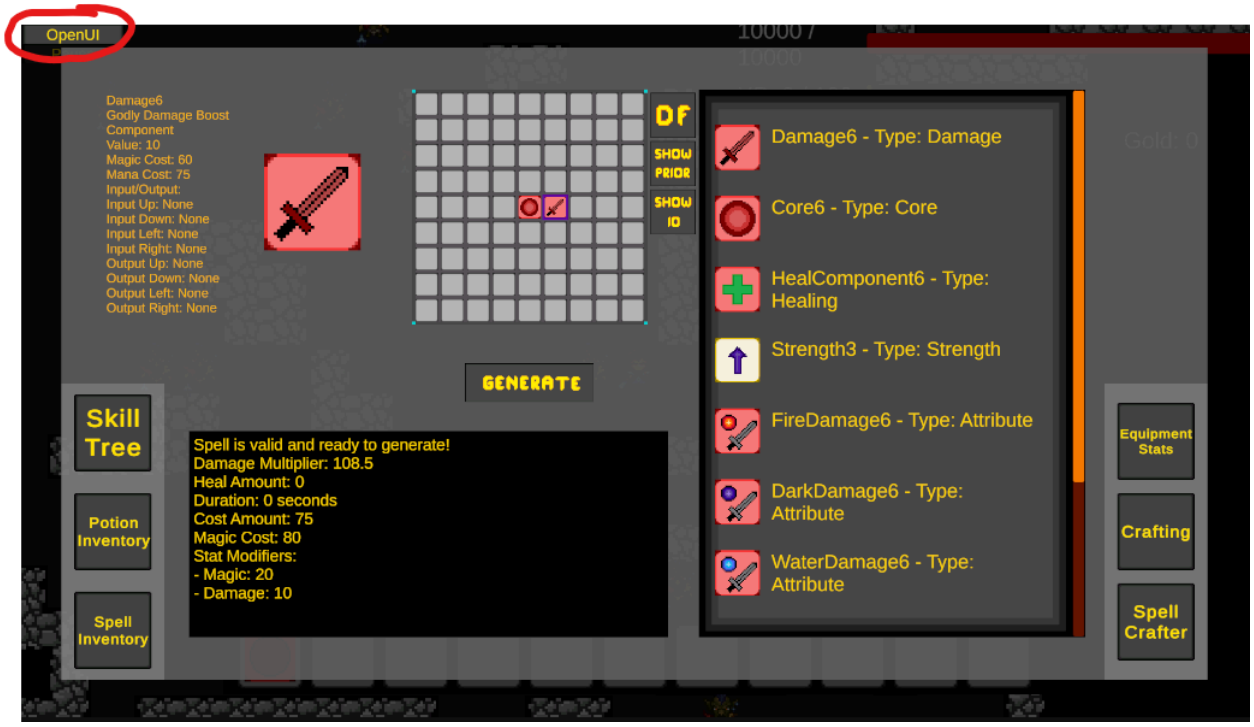
If your spell is valid, a description will pop up. This is only useful for regular components, it does NOT take into account DF components. Try placing a “Strength” component next to the “Damage” component.

To generate the spell, click the Generate button. This will only work if your spell is valid.

A menu will appear that allows you to name the spell. This is optional, the spell name does not really matter. Put whatever you want in it.

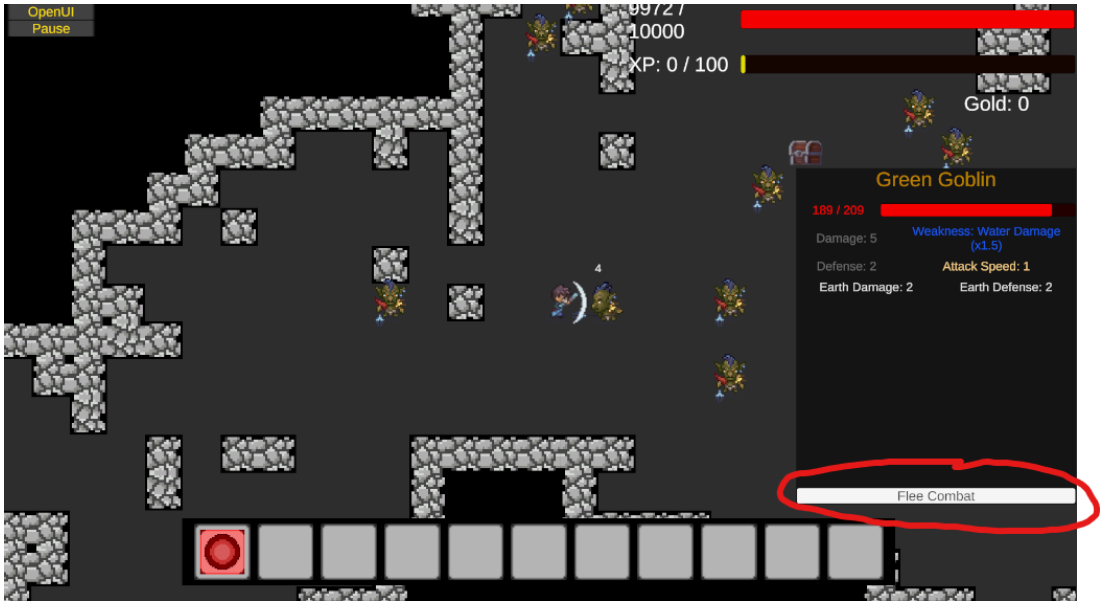
To finalize the spell, click the Submit button. Spells CANNOT be changed after they have been created.

Now, close the menu.



Now, you will see a core component in your hotbar. Clicking on this component will cast the spell. Each spell has a cooldown, during which it cannot be cast. Damage spells will be cast on your next attack.

To enter combat, just walk into an enemy. Press the W key to move up, the S key to move down, D to go right, and A to go left. Once you are in combat, you will automatically start attacking the enemy. Combat will continue until you kill the enemy. To exit combat early, press the Flee Combat button. Pressing the spell during combat will cast it. The damage spell you just made will instantly kill the enemy, assuming you used the same components as in this tutorial. A spell that contains a heal component will heal the player the moment you cast the spell.



To close and exit the program, press the Pause key or press Escape.

If you press the Resume button, you will unpause the game. If you press Quit, the game will close.

Basic Spell System Overview

Spells are placed on a grid. Each component can see it's direct neighbor, and activates based on that. There is a core component off of which the spell is created. All spells must have a core component. Spells do not have a mana cost, but rather a cooldown time. More components generally means a longer cooldown, but for testing purposes this feature has been disabled and all spells have a fixed cooldown. In the description, you can basically ignore the "Magic" and "Cost" fields, those are disabled for ease of testing. They do things not required by this assignment.

Damage: deals damage

Heal: heals the player

Core: core of the spell. Required for every spell.

Strengthen: strengthens the effect of the adjacent component. Can stack additively on affected components, but cannot affect other strengthen components.

Attribute damage: applies attribute damage

Dataflow Spell System Overview

Spells are built on a dataflow grid, which is a small 2D grid of tiles where you place components that pass signals to each other. When a spell is cast, the grid is evaluated left-to-right, top-to-bottom, in multiple passes. Components communicate by outputting signals through their active ports (Up/Down/Left/Right), which neighboring components read as inputs.

There are three signal types:

- Number: a float value (damage, health, etc.)
- Bool: true or false (conditions)
- Attribute: a stat type (e.g. FireDamage, WaterDamage)

Each component has input and output ports on specific sides. A component can only read a signal from a neighbor if both the input port on your side and the output port on the neighbor's side are facing each other.

Component Reference

Values / Sources

These produce signals with no inputs needed.

Constant Number
Outputs a fixed number you set in the inspector. Place it and configure the value via the number setter UI. Use it anywhere you need a hardcoded amount (e.g. "deal 10 damage").

Constant Attribute
Outputs a specific attribute type (e.g. FireDamage). Use it to tell Set Attack Attribute which element to apply, or to feed Equals Attribute for comparisons.

Enemy Weakness Sensor
Outputs the enemy's weakness as an Attribute signal. If the enemy has no weakness, outputs nothing. Use it to dynamically target the enemy's weak element.

Operators

These transform signals.

Equals Attribute
Takes two Attribute inputs and outputs a Bool: true if they are the same type. If the second input isn't connected, it falls back to a hardcoded compareTo attribute set in the inspector. Use it to check "does the enemy's weakness match a specific element?"

Bool Inverter
Inverts a bool, exports it in all directions other than the input.

Effectors

These are terminal nodes. They consume signals and produce the actual spell effect. They only run on the final evaluation pass.

Add Damage
Takes a Number input (the damage amount) and an optional Bool input. If the Bool is connected and false, no damage is applied. Otherwise the number is added to the spell's total damage. Has a mana cost multiplier based on the damage value. Mana cost is currently disabled for ease of testing.

Set Attack Attribute
Takes an Attribute input and a Number input. Sets that attribute's damage value in the spell's temporary attack attribute set. This is what makes a spell deal elemental damage (e.g. FireDamage: 15). The attribute damage is then applied through the normal enemy weakness/defense calculation in combat.

Heal
Takes a Number input and heals the player for that amount immediately on cast.

How Signals Flow

Signals travel between adjacent grid cells only. A component at position (2,1) can only receive input from (1,1), (3,1), (2,0), or (2,2). The output port of the sender and the input port of the receiver must both be active and face each other.

Example: "deal 10 fire damage":

Set Attack Attribute sits in the middle, reading the number from the Left and the attribute from the Right.

Example: "deal bonus damage if enemy is weak to fire":

Equals Attribute checks if the enemy's weakness is Fire, Gates the 20 damage behind that condition, and Add Damage fires only if it passes.

Bridge

Passes along inputs like booleans, floats, and attributes.

Cooldown

Cooldown scales with the number of components placed in the grid. More complex spells will generally have longer cooldowns. This has been heavily nerfed and in some cases, removed, for testing purposes.